

nessa



Non-interactive - Encrypted - Self-Sovereign - Session-Auth - Attestation

1-2: Problem

Modern applications accumulate sensitive cryptographic "breadcrumbs".

- Key rotations, device enrollments, policy changes, proof-of-possession events.

These facts are spread across logs, tokens, time-stamps and stores etc.,

Verifying them later is brittle, chatty (leaky) and privacy-eroding:

- Verifiers usually need raw logs, linkable attestations etc.
- Users leak timing, ordering, identity metadata.

General ZK prover/verifiers are too generalized and complicated to integrate.

2-2: Problem

Dev teams across the world are facing a strategic dilemma:

- Ship on elliptic curve cryptography today and risk a painful post-quantum migration later.
- Bet early on post-quantum cryptography and pay performance and tooling costs now.

Solution

Pain Alleviated

- Stop sharing your entire bank statement just to prove one fact.
- Stop being a data honeypot. No more storing toxic, raw user data for audits.
- Stop choosing between security and privacy. Get both.
- Stop slow, invasive "spot-checks." No one needs to see your private data.
- Stop leaking metadata. Verification doesn't reveal what was checked.
- Stop trusting, start verifying. Eliminate the risky third-party verifier.

Pleasure Provided

- Prove, don't show. Share a tiny proof (under ~20 KB), not your sensitive data.
- Compliance without sniffing. Verify 100% of events with zero privacy cost.
- Your device, your rules. All checks happen locally. Nothing leaves.
- Mathematical certainty. Get cryptographic proof that a policy was met.
- One proof, infinite events. Compress thousands of logs into a single "pass" in $O(n \log n)$ time (proof gen).
- Instant, private access. Log in securely without revealing anything about yourself.
- Dual engine - one design: ECC+PQ

NESSA

- **NESSA** (also **nessa**) is a **privacy-first** , **self-hosted** cryptographic substrate for managing identity, access, and credential flows.
- It enables generation/attestation of a succinct proof for the verifier that reveals only what a verifier needs.
- It is the **collective operating model** , including policies, lifecycle management, and APIs, that operationalizes the “**qfold**” technology to provide **privacy by default** .

qFold

- It is cryptographic folding transform that aggregates many event commitments into a single, compact commitment and an accompanying succinct proof of policy-compliant correctness.
- state change $\rightarrow$ fold fact into accumulator $\rightarrow$ proof-of-well-formedness
- Verifier can then check the one object, learns nothing else, and remain agnostic to underlying cryptography.

Future Proof Security

NESSA aims to maintain two compatible backends:

- qfold-ec (Elliptic Curve) is built on Pedersen-style commitments + folded inner-product arguments (**pre-funding, MVP track**).
- qfold-pq (Post-Quantum) is built on lattice-based (Module-LWE) commitments and folded lattice sigma-protocols. It is NIST-aligned and provides a migration path for quantum resilience (**post-funding, production track**).

Business Model

NESSA is fully open-source and based on reputed and highly cited research papers (and will remain as such, forever).

Hyperversal Blocks (or HBLOCKS, the dev team working on NESSA) will build sandbox digital-wallets, FATF pilots, key managers, and domain-centric SDKs.

These will be offered as standalone commercial products, alongside enterprise-level support, to drive mass adoption and fund the protocol's development.

Competitive Analysis

StarkWare: Product: ZK-STARKs, StarkNet (L2)

- Valuation: \$8 Billion as of May 2022 (private)
- Liquidity: Market-cap of ~\$535M with ~\$1.24B FDV
- Niche: NESSA is local-first ZKP user, StarkNet is an “on-chain” ZKP inventor/researcher goliath.

Matter Labs: Product: zkSync (L2)

- Valuation: ~\$721M with \$458M in funding
- Revenue: ~ \$35.7K 30-Day revenue
- Niche: NESSA is application layer, zkSync is at infrastructure layer.

Polygon: Product: Polygon (PoS, zkEVM), Polygon ID

- Valuation: ~\$20B with \$450M in recent funding
- Niche: Polygon is both infrastructure and application layer/provider.

Use Case 1: Proving Bank-Wide AML Compliance to Regulators

Imagine a national bank needs to prove to its government regulator (who, in turn, reports to FATF) that it is fully compliant with anti-money laundering (AML) policies for millions of its customers.

Use Case 1: Proving Bank-Wide AML Compliance to Regulators

The problem

The regulator must conduct **slow**, **expensive** and **privacy-invasive** spot-checks.

They might request the full **KYC files** for thousands of specific customers, copies of **passports**, **bills**, and **transaction histories**.

This is a massive **data liability for the bank** and a **slow process for the regulator**.

Use Case 1: Proving Bank-Wide AML Compliance to Regulators

The Solution

- **Event Collection:** For each of its millions of customers, the bank has a list of sensitive "compliance events" it has performed as required by FATF:
 - Event 1: Customer ID successfully verified (KYC).
 - Event 2: Address successfully verified.
 - Event 3: Source of wealth check completed.
 - Event 4: Politically Exposed Person (PEP) screening completed.
 - Event 5: Daily ongoing transaction monitoring since last year.

Use Case 1: Proving Bank-Wide AML Compliance to Regulators

The Solution

- **qFold Compression** : Instead of storing/sharing raw files, the bank uses “nessa” (powered by qfold) to compress the millions of events (multiple events per millions of users) from across its entire customer base.
- **Verifiable Object (Proof)** : nessa outputs a single, succinct proof of well-formedness.
- **Disclosure** : The bank hands this one small proof to the regulator.

Use Case 1: Proving Bank-Wide AML Compliance to Regulators

The Result

The regulator can instantly verify this proof (using nessa verifier sandbox) and confirm (with mathematical certainty), that the bank has successfully completed all 5 required compliance checks for all of its millions of users.

The regulator **learns** that bank is 100% compliant but they do **not learn** any private details,

- No names, no address, no account number or passport information.

This allows for real-time, total **regulatory oversight** while perfectly **preserving** customer **privacy** .

Use Case 2: Verifiable Accredited Investor Status

The Problem

To use certain investment platforms, a person must prove they are an "accredited investor".

- Prove a net worth over \$1M.
- Income over \$200K.

To do this, they must upload sensitive documents like tax returns, and bank statements to a third-party verifier.

This creates a major privacy risk and a data honeypot.

Use Case 2: Verifiable Accredited Investor Status

The Solution

- **Event Collection:** User runs a nessa-prover “wallet” or agent locally on their own device. This agent securely connects to their financial accounts (bank, brokerage, etc.,) and nessa-verifier using APIs.

Agent performs all the checks (sub-events) privately on user’s device e.g.

- a. $\text{Account A} + \text{Brokerage B} = \X (Policy: Check passed)
- b. $\text{Sum Digital Asset C} = \Y (Policy: Check passed)
- c. $\text{Liabilities (mortgage, loans, credit lines)} = \Z (Policy: Check passed)
- d. Final Policy Check: $(\$X + \$Y - \$Z) > \ACCREDITED_REQ (Result: True)

Use Case 2: Verifiable Accredited Investor Status

The Solution

- **qfold** : The agent takes these sensitive financial events and transforms them using qFold into a single, verifiable cryptographic proof.
- The proof essentially just says,
 - I have successfully run the 'Accredited Investor' policy check
 - Result is true
- Disclosure: The user presents *only this single, tiny proof* to the investment platform.

Use Case 2: Verifiable Accredited Investor Status

The Result

- The investment platform knows with mathematical certainty that the user is an accredited investor.
- The platform learns nothing about their actual net worth, their income, what banks they use, or what assets they own.
- The user gains access without ever revealing any underlying private data.

Road to MVP

- Technical whitepaper: ✓
- ReadMe + Deck: ✓
- Website: 🚧
- Mocks: 🚧
- Auth/oauth Sandbox: 🚧
- Audits: ✕

Ask: ~\$10000-\$15000 (primarily to bootstrap prototype) to be used to cut down lead time and on-board more devs.

Grants and funding to be used to further the development of sandboxes and protocol.

Financial Strategy: Grants/Funding

Auth/oauth Sandbox: ~3 Months

FATF compliance test-sandbox: ~5-6 Months

Lead times will improve with addition of more team members.

TEAM



Iqra

Founding Designer
UI/UX

[Linkedin](#)



Haseeb

Founding Dev
Research/Protocol
(1x DAI 10K Grant from [EthSwarm](#))

[Linkedin](#) | [Git](#) | [X](#)



Haider

Founding Dev
Full-Stack/FE

[Linkedin](#) | [Git](#)

Important Links

Website: <https://nessa.sh> (WIP)

Contact: +923168021991

Email: has33braja@gmail.com

- Hyperversional Blocks [git repo](#).
 - Technical [Whitepaper](#).
 - Deep-dive into non-tech [Readme.md](#).
 - Author [Linkedin](#).
 - Research papers can be furnished on request.
-